技术白皮书

2026 年 10 月

岩芯矩阵

Ultra-Fast Token

从 KV Cache 不增长，到单用户百万 token/s

World-State EdgeRock DiffusionRock SiliconRock



技术摘要

强推理，省显存，快生成。

模型正在从回答问题走向按需生成软件。用户提出一个需求，模型就能当场写出对应的 Agent、代码和工具，软件从此可以随用随生。这时候，等生成就成了新的门槛：一份可用的 Agent，背后可能要生成数万乃至数十万 token 的程序、工具接口和测试代码。按今天单用户每秒一两百 token 的速度，用户往往要等上十几分钟；把速度推到每秒百万 token，按需生成就能进入实时交互。所以，生成必须更快，而且要快的是单个用户的生成速度，好让软件和 Agent 更早用上。

要让生成快起来，第一道坎是记忆。主流大模型用 KV Cache 保存历史，推理越长，占用的显存就越多，每生成一个 token 要读的历史也越多；于是，一张卡能同时服务的请求越来越少，单个请求也越写越慢。业界常见的办法是压缩，让每个 token 存得更小、读得更少。然而，只要缓存还在随推理增长，压缩就只是把内存墙往后推。岩芯矩阵（Rock Matrix）换了一个思路：不压缩 KV Cache，而是让它根本不增长。

为此，我们提出了世界状态（World-State）：模型不再保存一份越写越长的完整历史，而是维护一份容量固定的状态，用它概括当前任务和环境，并随新信息的到来不断更新。固定容量的记忆并不新鲜，真正的难点在于，记忆一旦固定，模型往往会忘掉远处的关键细节。我们的答案是记忆岩锚（Rock Memory Anchors，RMA）：模型自己学会判断哪些细节值得长期保留，并像攀岩者钉下岩锚一样把它们固定住。原文离开局部窗口后，后面的推理仍能读到留下的细节。有了记忆岩锚，首个开源模型 EdgeRock-4B 在 256K 上下文处减少 99.1% 的注意力 KV，AIME 2024–2026 平均得分仍达到 82.9%。而且，每条请求的内存固定以后，同一张 GPU 能同时服务更多请求，长推理吞吐提升到来源模型的 6.42 倍，每个 token 的服务成本随之下降。

吞吐上去了，是省；可对单个用户来说，模型还是一个 token 一个 token 地写。DiffusionRock 换了个写法：先一次写出一整块草稿，再用几轮集中修改没把握的地方，每读一遍权重，整块一起更新。这样一来，单请求速度已达每秒 1,500 token，比今天常见的速度快了一个数量级，而且内存同样不增长。

不过，只要还跑在 GPU 上，每一轮仍要从显存里读一遍权重；整块生成能分摊这笔读取，却去不掉它。SiliconRock 把主权重直接做成电路，生成时不必再从系统内存或昂贵的 HBM 里读取，固定容量的状态则放进片上 SRAM。这样做，首先是省：主路径上不再需要 HBM，封装和互联也跟着简化，每个 token 的成本也降了下来。其次，不只是省，还更快：权重不用搬，状态就在计算单元旁边，完整的生成在一块芯片里就能走完，路径更短，生成也更快。既省又快，是我们走向 Ultra-Fast Token、单用户每秒百万 token 的基础。到了这个速度，模型不仅能当场写出软件，还能实时推演虚拟世界，直接控制机器人。

三项关键技术

固定记忆容量
维护任务信息，由记忆岩锚保留关键历史。

整块生成内容
整块生成并集中修正，减少串行等待。

权重不从外存读取
主权重固化成电路，状态留在片上 SRAM。

核心主张

强：保留强推理能力。

省：降低每个 token 的成本。

快：目标单用户百万 token/s。

产品交付形态

EdgeRock
固定记忆，多模态理解与推理，输出文本。

DiffusionRock
超高速生成：单请求已达每秒 1,500 token，正向 any-to-any 多模态生成扩展。

SiliconRock
全模型固化进单芯片，主路径无需 HBM，既省又快。

为什么生成必须更快

当软件按需生成，等待生成就成了新的使用门槛。

过去，软件要先由人写好，再交给用户。现在，用户刚提出需求，模型就能当场写出对应的 Agent、代码和工具组合。这样一来，许多原本不值得专门开发的软件，都可以随用随生。不过，这有一个前提：生成必须在一次交互里完成，否则用户就只能等着。

这里最关键的是解码速度，也就是模型为一个用户连续写出 token 的速度。模型读完需求，还要写出推理、程序和结果，写得越长，用户等得越久。Agent 尤其如此：交给人的也许只有几百字，背后却要反复规划、写代码、调用工具、读取结果、检查和修正，一项任务动辄生成数万乃至数十万 token。这些 token 大多不是写给人看的，而是 Agent 自己用来思考和行动的。所以，快的意义不在于人读得更快，而在于软件和 Agent 更早可用。

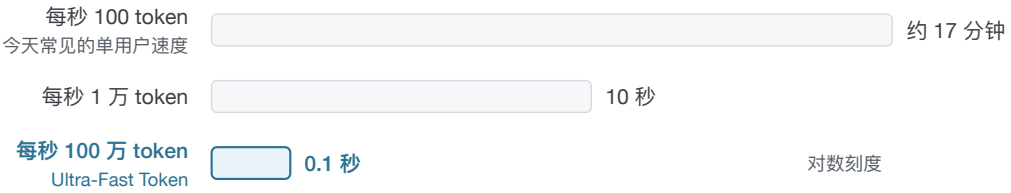


图 1. 生成 10 万 token：100 token/s 需约 17 分钟，百万 token/s 仅需 0.1 秒。

速度还决定了模型能被用得多频繁。持续的屏幕理解、现场分析和环境观察，需要模型不停地更新判断；交互式数字环境需要随用户操作及时变化，世界模型需要在有限时间内推演不同选择的结果，机器人则要根据新观测及时生成和修正动作。生成足够快，通用模型的理解与推理才能持续参与交互和控制。

快模型和更强的大模型并不冲突，而是互相补位：前沿大模型负责把能力的上限往上推，快模型负责把成熟的能力带进高频、持续的使用。岩芯矩阵做的是后者，让强模型以单用户百万 token/s 的速度持续生成，我们把它叫作 Ultra-Fast Token。这个目标，我们分三步来实现。

岩芯矩阵概览

单个用户的生成速度被三道墙挡着，我们分三步，一步拆掉一道。

第一道墙是记忆。主流大模型把历史存进 KV Cache，推理越长，存得越多，每一步要读的也越多。第一步，EdgeRock 让记忆的大小固定下来。推理再长，内存也不增长，推理能力也不打折。

第二道墙是串行。模型一次只写一个 token，写完一个，才能写下一个。第二步，DiffusionRock 改成一次写一整块，再用几轮修改把它写好，要等的轮次就少了很多。它的内存同样不增长。

第三道墙是搬运。在 GPU 上，模型每写一轮，都要把全部权重从显存里读一遍。第三步，SiliconRock 把权重直接做成电路，把状态放在芯片里。权重不用再搬，整个生成在一块芯片里完成。

三步一步比一步快，目标是单用户每秒百万 token。EdgeRock 和 DiffusionRock 都是完整的模型，可以单独用，也可以组合起来用。

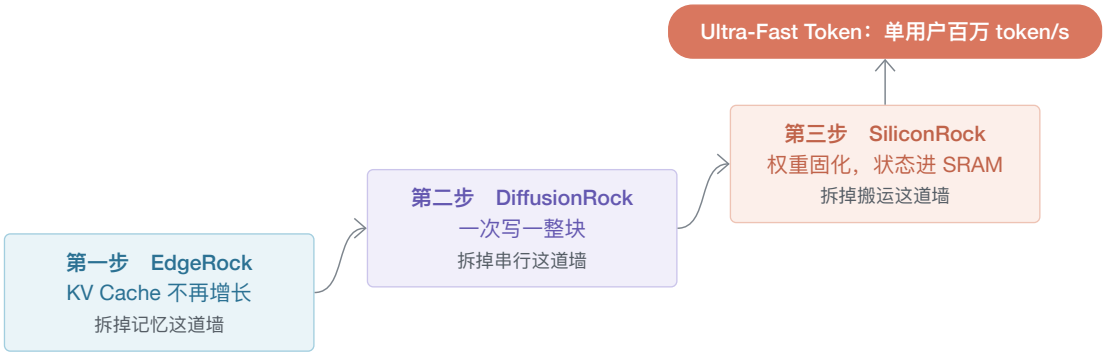


图 2. 三步走向 Ultra-Fast Token，每一步拆掉一道墙。

要点

- **不增长**：KV Cache 大小固定，推理再长也不多占显存。
- **强**：内存固定，推理能力不打折。
- **省**：同一张 GPU 能服务更多请求，每个 token 更便宜。
- **快**：单个用户的生成速度提升一个数量级。
- **端云通用**：EdgeRock 既能部署在数据中心，也能跑在端侧设备上。
- **已开源**：EdgeRock-4B 的权重和推理代码已经发布。

固定记忆的设计原则

这三步有一个共同的基础：固定记忆。它要同时做到三件事：历史内存不增长，关键细节留得住，省下的内存能转化为速度。世界状态是模型内部保存任务与环境信息的记忆，RMA 负责筛选并保留其中的关键历史。

原则一：KV Cache 不增长

不给缓存瘦身，而是让内存在设计时就定下来。

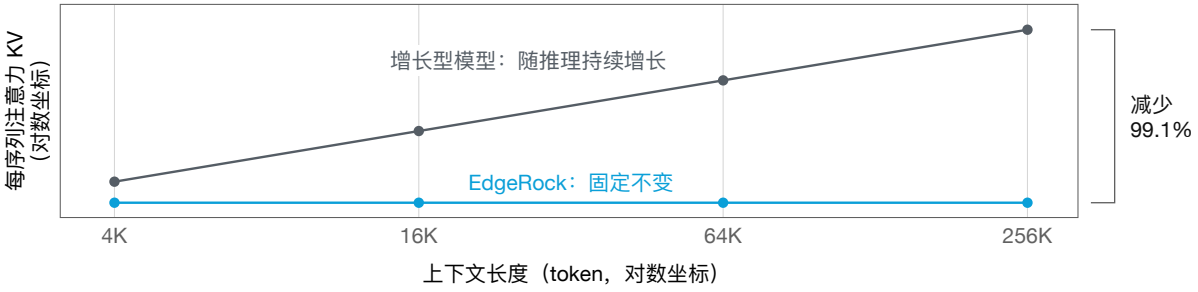


图 3. EdgeRock 的 KV Cache 不随上下文增长：在 256K 上下文处，比来源模型少 99.1%。

今天的主流做法，是让每个缓存的 token 变小、变少：多头潜在注意力把每一项压小，稀疏注意力只读其中一部分，量化降低精度，Qwen3.5、Kimi Linear 这类混合架构只在四分之一的层保留全注意力。这些办法让每个 token 更省，可缓存还是随着每个 token 增加；放慢增长，只是

把内存墙往后推。岩芯矩阵则从结构上取消了增长：模型的历史放进一份容量固定的状态，4K 和 256K 的推理用的是同一份内存。部署要的是确定性，而不只是效率。

原则二：推理能力不打折

办法是只记值得记住的。

固定内存的模型并不少见，可它们要么把历史压成一份模糊的概要，要么一出窗口就忘，后面推理要用的细节找不回来，推理能力也就跟着往下掉：我们评测过 16 个这类有界状态模型，最大的有 14B，AIME 2024–2026 最高也只有 18.8%。世界状态把历史分成三部分：递推状态抓大意，局部窗口管眼前，锚点库记关键。前两部分沿用业界成熟的 Gated DeltaNet 和滑动窗口注意力；真正决定记什么的，是我们提出的记忆岩锚（RMA）。它每处理 256 个 token 取舍一次：为新候选学习重要性分数，再和已有锚点按分数筛选，每个 KV 组只留下得分最高的 1,024 条。已有锚点沿用写入时的分数，留下的记录内容不变，原文滑出窗口以后照样能直接读到；而且每条都标着来源位置，模型记住了什么，可以直接查看。只留关键、不留冗余，内存固定了，推理照样强。

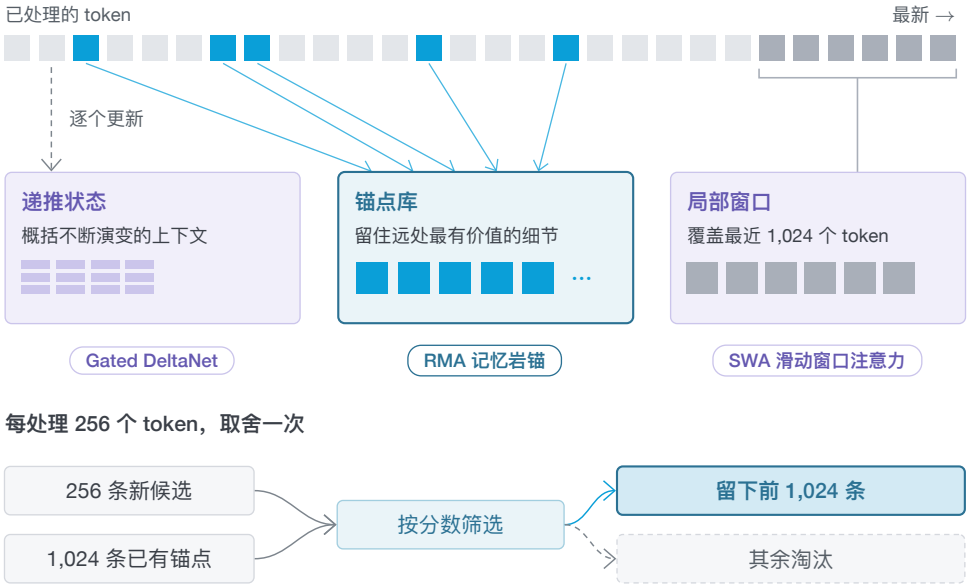


图 4. EdgeRock 的固定记忆：递推状态概括上下文，局部窗口保留近期内容，RMA 锚点库留住关键细节。

原则三：存算联动

省下的内存，在 GPU 上变成吞吐，在芯片上变成速度。

在 GPU 上，固定记忆给每条请求的历史存储和读取开销封了顶，同一张卡可容纳更多并发请求，吞吐提高，单位 token 的服务成本随之降低。对于专用硬件，确定的状态容量也让片上 SRAM 可以按需配置，读写就在计算单元旁边完成。固定记忆由此同时改善内存使用和数据访问。



图 5. 固定记忆的两类收益：GPU 上提高吞吐、降低成本；专用芯片上缩短数据路径、加快生成。

EdgeRock：恒定内存下的强推理

理解多模态，输出文本，持续推理更省显存。

EdgeRock 采用世界状态记忆结构，侧重长任务的持续理解与推理。开源的 EdgeRock-4B 由 Qwen3.5-4B 转换而来：保留 24 层 Gated DeltaNet 递推层，把其余 8 层全注意力换成 1,024 位置的滑动窗口注意力（SWA），再加入记忆岩锚。训练先让锚点模块学会存取，再与主干联合训练，随后通过监督后训练和可验证奖励强化学习打磨推理能力。这条转换路线正在推广到更多架构；同时，我们也在开展从零预训练，让模型从预训练开始就使用固定记忆。

恒定内存和强推理可以兼得。EdgeRock-4B 在 AIME 2024–2026 上的平均一次通过率是 82.9%，和内存随推理增长的领先小模型处在同一水平，是本次评测中有界状态模型最好成绩的四倍多。这份能力正是锚点带来的：推理时把记忆岩锚全部关掉，长上下文里的远距查询正确率会从 94.1% 跌到 0.4%。图 6 给出了数学、知识、科学和代码的四项对比。

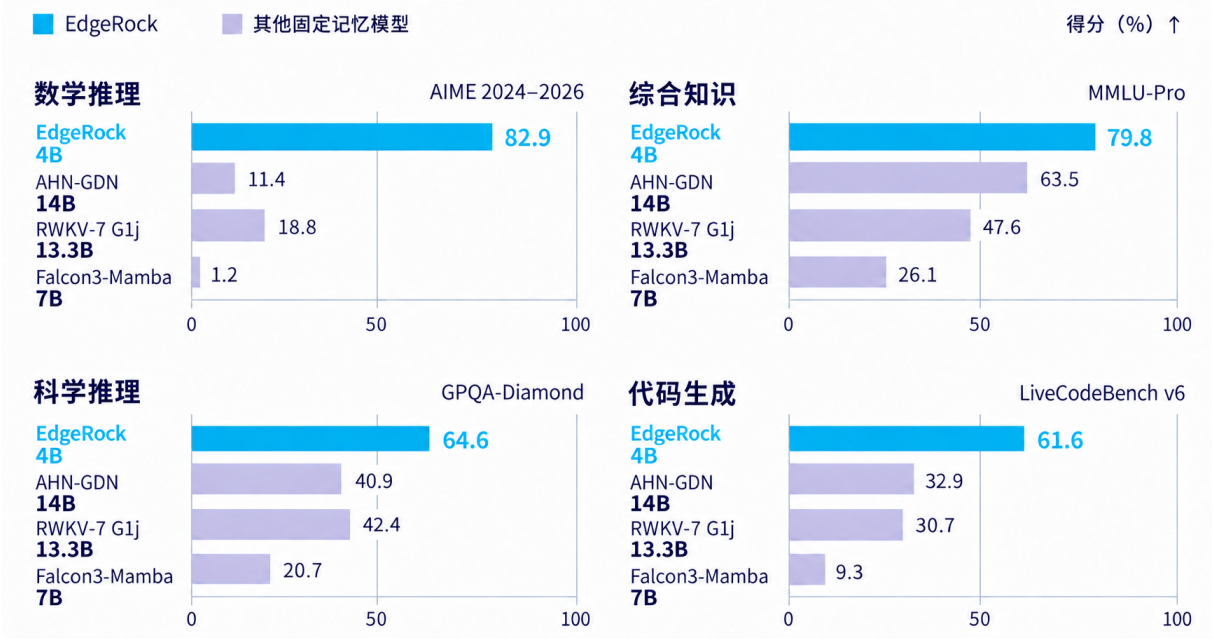


图 6. EdgeRock 仅用 4B 参数，四项能力得分均领先图中 7B 至 14B 的更大固定记忆模型。

当前模型支持文本与图像输入，输出文本、代码和结构化结果，适合持续对话、长文档分析与长期 Agent 任务；视频、音频和环境信号的理解是后续扩展方向。

性能：推理越长，不再越贵

内存固定下来，好处直接体现在服务上。先看显存：一张卡同时跑 512 条长推理（每条 32K 输入、32K 输出）时，EdgeRock-4B 的显存峰值约 61 GiB，连模型权重都算在内；如果让来源模型也同时跑这 512 条序列，光是 KV Cache 就要约 1 TiB，相当于 13 张 80GB 显卡。EdgeRock 一张卡就装下了。

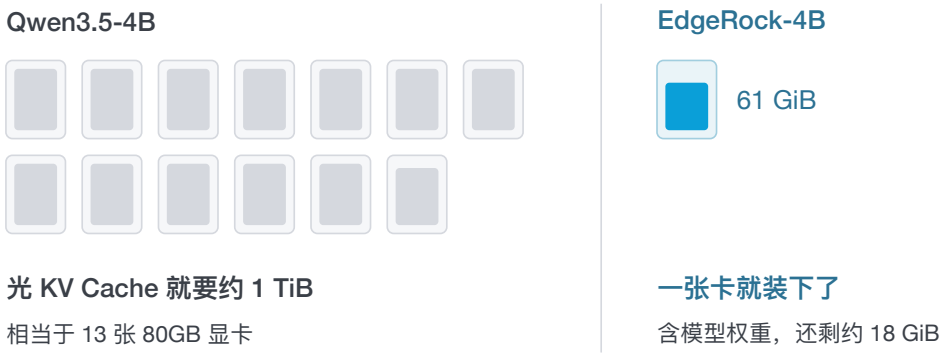


图 7. 512 条长推理同时跑：来源模型光 KV Cache 就要约 13 张 80GB 显卡，EdgeRock-4B 连权重在内一张卡就够。

再看吞吐：一张卡装得下更多请求，吞吐自然就上去了。同样的 GPU 租用时间里，吞吐翻几倍，单位 token 的 GPU 成本就降到原来的几分之一；而且上下文越长，固定记忆的好处越明显。

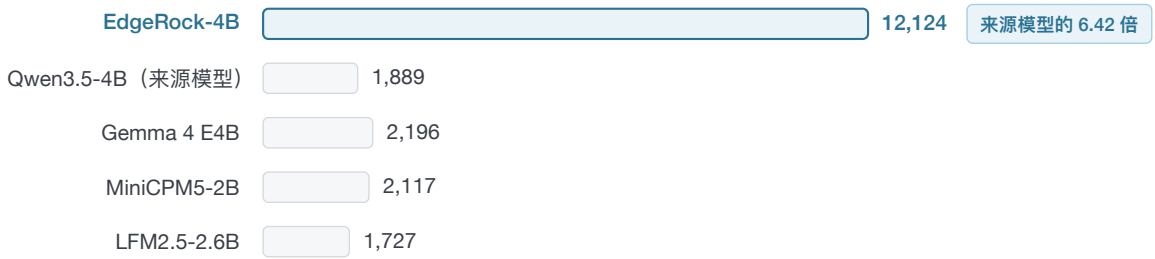


图 8. 同一张 GPU，EdgeRock-4B 的长推理吞吐是来源模型的 6.42 倍（32K 输入、32K 输出，单位 token/s）。

吞吐决定服务成本，单用户速度决定等待时间。EdgeRock 的单请求解码速度也是来源模型的 1.54 倍；要让一段代码或一份计划更早写完，还可以从生成方式本身入手。

DiffusionRock：面向 any-to-any 的高速生成

一次写一整块，单请求已达每秒 1,500 token。

快在哪里

DiffusionRock 的第一个版本以 DiffusionGemma 为基座。和今天常见的每秒一两百 token 相比，它让单个用户快了一个数量级。

关键在读权重的次数。今天主流的大模型，比如千问（Qwen）和 DeepSeek，都是自回归模型：每个新 token 都要根据前面写好的内容来生成，所以只能一个一个地写。每写 1 个 token，都要把权重从显存里读一遍，写 256 个 token 就要读 256 遍。DiffusionRock 换了个写法：先一次写

出 256 个 token 的草稿，再用少量几轮集中修改没把握的位置。每读一遍权重，256 个位置一起更新，几轮就能写完一整块。读权重的次数少了，单用户生成时原本闲着的算力也用上了，速度自然就上去了。而且，这是在内存不增长的前提下做到的：DiffusionRock 同样用记忆岩锚管理历史，任务再长，历史 KV Cache 也不变。

DiffusionRock 本身就是一个完整的模型：给它需求和上下文，它就能直接写出回答、代码或工具参数。它和 EdgeRock 的区别在于写法：EdgeRock 一个一个写，擅长长推理；DiffusionRock 一块一块写，专攻速度。

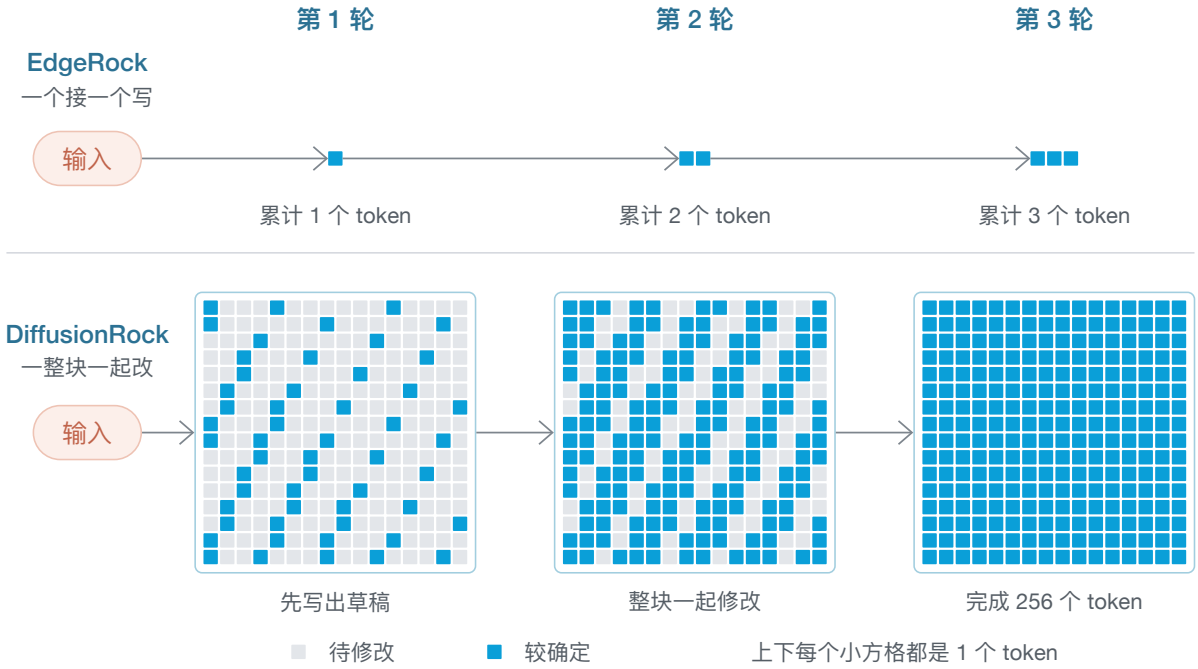


图 9. 同样走三轮，EdgeRock 只写出 3 个 token，DiffusionRock 已经写完一整块 256 个 token。

走向端到端的世界模型

接下来，DiffusionRock 会从文字和代码扩展到图像、音视频和动作，走向 any-to-any 的输入输出。要成为世界模型，它还得学会环境会怎么变化、行动会带来什么后果，进而生成未来的场景或候选动作，用在数字交互、虚拟环境、工业仿真和具身智能里。

再往前一步，我们打算把理解和生成放进同一个模型，联合训练。拿 NVIDIA 面向物理 AI 的世界模型 Cosmos 3 来说，它本来就由两个分支组成：理解分支是自回归模型，逐个生成 token，负责看懂指令和画面、完成推理和规划；生成分支是扩散模型，整块生成画面、声音和动作。我们让理解分支采用 EdgeRock 的架构，生成分支采用 DiffusionRock 的架构，两侧的记忆都由 RMA 固定。这样，新的观测一进来，模型就更新记忆、规划下一步，再生成新的动作。



图 10. 以 Cosmos 3 为例：理解分支采用 EdgeRock，生成分支采用 DiffusionRock，记忆都不增长。

模型与芯片，一起设计

从第一性原理出发：权重不搬，状态不长，完整生成留在单芯片。

说回速度。前两步都还跑在 GPU 上：固定记忆让历史不再越读越多，整块生成让串行的轮次少了很多，可每写一轮，仍要把整个模型的权重从显存里读一遍。以一个约 21 亿参数的模型为例，即使权重压到 4 位，逐 token 生成每一步也要搬约 913 MB；要每秒一百万 token，就需要每秒 913 TB 的带宽。而 NVIDIA 最新一代的 Rubin GPU，显存每秒也只能读约 22 TB，单从理论上算就慢了四十多倍，怎么优化也做不到。整块生成能把这笔开销分摊到很多 token 上，却去不掉它。

推理时不变的权重，就不必每轮读取；必须更新的状态，就留在计算单元旁边。这两件事要同时做到，光改芯片不够。Taalas HC1 已经把权重固化进单芯片，不再依赖 HBM，KV Cache 放在片上 SRAM 里，跑 Llama 3.1 8B 的单用户速度约每秒 1.7 万 token。但 Llama 的 KV Cache 随上下文增长，上下文越长，要的 SRAM 越多，每一步要读的也越多。反过来，只改模型、不改芯片，KV 不再增长，权重却仍要搬。只有模型和芯片一起改，两项搬运才会同时消失。



图 11. 只改一边只能去掉一半，两头一起改，两项搬运才同时消失。

这样做既省又快。省，是因为主路径上不再需要昂贵的 HBM，封装和芯片间的互联也简单了，整套系统精简下来，每个 token 的成本自然更低；快，是因为生成不再等权重从外面搬进来，整条路径在一块芯片里就能走完。

一起设计，也不等于一切从头来。我们只重做随上下文增长的那部分记忆，成熟模型已有的语言、推理和多模态能力都保留下来，开放模型的进步也能持续吸收进来。

SiliconRock：把模型完全固化进芯片

整条生成路径都在一块芯片里走完，每个 token 的时间表排在 1 微秒以内。

SiliconRock 把定型的模型整个做进一块芯片：从读入 token、逐层计算、更新状态，到选出下一个词，都在片内完成，芯片对外只收发请求和结果。独立模型和联合训练后的组合模型，都沿这条路线走向单用户百万 token/s。这一部分，我们正在寻求合作伙伴共建，一起把模型做进芯片。

权重变成电路，状态留在身边

在 GPU 上，权重存放在显存里，每生成一个 token 都要整个读一遍，计算单元常常在等数据。SiliconRock 反过来，把每个权重直接做成电路的一部分：数据流过时，乘法就在权重所在的位置完成。生成时权重一个字节也不用搬，既不从 HBM 读取，也不用先搬进 SRAM。

SRAM 留给状态。世界状态的大小是固定的，上下文再长也不变，所以可以整个放进片上 SRAM，按层存放在计算单元旁边。每生成一个 token，各层就近读写状态，层与层之间只传当前 token 的计算结果。

连选下一个词也是就地完成的，有点像比赛先分组初赛、再进决赛：十几万个词的词表分成许多小组，每组就近选出几个最有希望的候选，再层层汇总，最后只在几百个候选里决出下一个词。



图 12. 一个 token 在 1 微秒内走完全程：权重就在电路里，状态就在计算单元旁边。

1 微秒走完一个 token

单个用户的生成是一场接力：上一个 token 走完全部 28 层、选出来、再送回输入，下一个 token 才能出发。每秒百万 token，意味着每一棒只有 1 微秒，在 1 GHz 时钟下就是 1,000 个时钟周期。我们就按这本账排了时间表：每层分到 30 个周期，乘加和状态更新都在这 30 个周期里重叠完成；28 层走完，再加上选词、采样和送回输入，一共 884 个周期，还在 1,000 个周期以内，折合每秒约 113 万 token。能排进 1 微秒，靠的正是前面两点：权重不用搬，状态就在旁边。

原型进展

我们已经在一个早期的恒定内存模型上做出了原型：完整模型的权重全部映射成了电路，从模型到芯片版图的全流程已经走通，电路的正确性也通过了测试。这个原型一个一个地写 token，用来验证固定记忆和权重固化这条路走得通。下一步，先在商用 EDA 工具上完成验证，再流片；随后再为 DiffusionRock 设计整块并行的调度，把两条提速路径结合起来。

固化不等于僵化

主权重和计算结构固化在芯片里；LoRA、行业词表、机器人的动作校准和安全阈值，则放在可以更新的部分，行业适配照样可以持续进行。模型还在快速迭代时，先在 GPU 上服务；能力定型以后，再做成 SiliconRock 芯片。

应用与部署

从按需生成软件，到推演虚拟世界，再到直接控制机器人。

对开发者来说，接入方式不变：岩芯矩阵的模型通过 OpenAI 兼容 API 等主流接口接入，原来的提示词、工具和工作流照常使用，换成更快的服务也不用重写应用。应用则随模态一层层往外扩：先是文字和代码，再到图像、视频和声音，最后是机器人的动作。

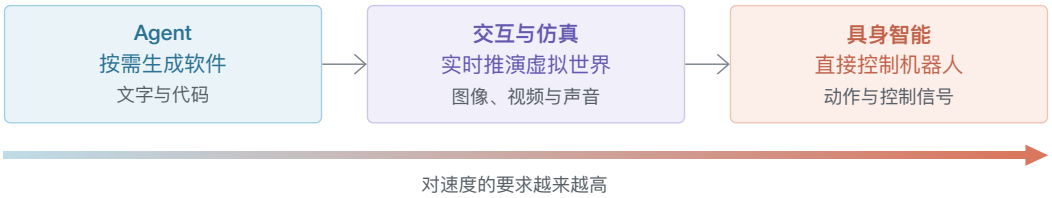


图 13. 从生成软件，到推演世界，再到控制机器人，越往后，对速度的要求越高。

Agent：与前沿大模型混合加速

在复杂 Agent 任务中，前沿大模型负责规划与校验，岩芯矩阵的快模型承担代码、工具参数和 Agent 程序的大量生成，再把结果交回校验环节。评估这一分工，要看整个任务的完成时间和成本。



图 14. 按需生成：前沿大模型负责规划与校验，岩芯矩阵快模型承担代码、工具接口和 Agent 的生成。

交互与仿真：让模型在环境中试验

世界模型本身就是一台仿真器：给它当前的画面和一个动作，它就生成接下来的画面。以 Genie 3、Cosmos 3 为代表的世界模型，已经能实时生成可以交互的虚拟世界，用来训练机器人和自动驾驶。不过，这类模型都卡在两件事上：一是速度，每一步都要立刻算出下一帧；二是记忆，交互越久，要记住的内容越多。

这两点，正是岩芯矩阵要解决的。接下来，我们会把世界状态用到交互式环境里，让内存不随交互增长；再用 DiffusionRock 整块生成画面，用 SiliconRock 把模型做进芯片，让下一帧来得更快。这样，机器人可以先在仿真里试动作、看后果、调策略，再到真实设备上执行。

具身智能：大模型直接控制机器人

今天的机器人，大多是“大脑”加“小脑”：大模型当大脑，负责看懂任务、做规划；真正发出控制信号的，是一个小策略模型，也就是小脑，像学生一样从大模型这个老师那里蒸馏而来。之所以另做一个小模型，是因为控制频率很高，大模型太慢，跟不上。可是小模型见得少，换个场景、换件物体就容易失手，大模型的泛化能力在这一步打了折扣。

近期的实验说明，用通用大模型直接控制机器人，效果反而更强：GPT-6 Astra 不做专门微调，表现就超过了专门训练的机器人策略。

它还差的，是速度。如果大模型自己就快到能跟上控制频率，就不必再蒸馏一个小脑：像 Cosmos 3 这样的大模型可以直接控制机器人，大脑和小脑合二为一，泛化能力也就直接用在了控制上。这正是 Ultra-Fast Token 在具身智能里最核心的用处：DiffusionRock 一次生成一整段动作，SiliconRock 把整个模型做进芯片，让大模型的速度跟上控制频率；固定记忆则让它长时间运行，内存也不增长。

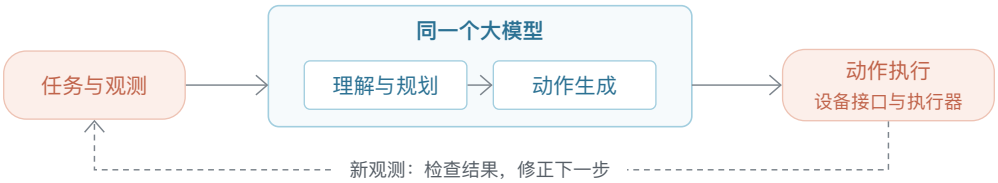


图 15. 大脑和小脑合二为一：同一个大模型理解任务、生成动作，并根据执行反馈持续修正。

路线图

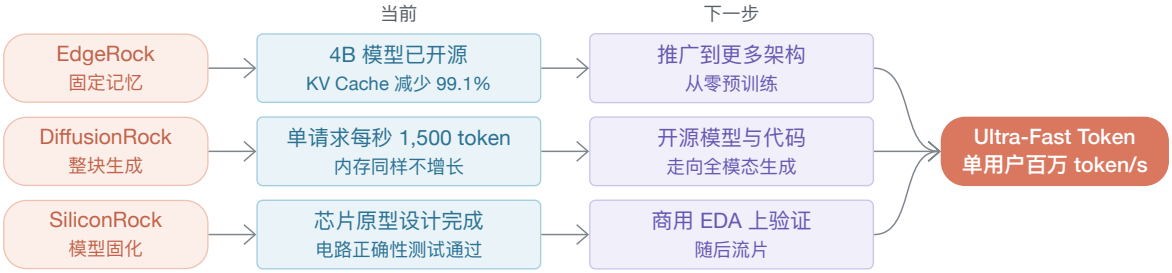


图 16. 三条路线各自向前推进，最终汇合到 Ultra-Fast Token。

结语

从按需生成软件，到世界推演与具身控制，岩芯矩阵沿固定记忆、并行生成与模型固化三条路线，走向单用户百万 token/s，让通用模型的理解、推理和行动跟上真实交互的节奏。

模型与代码 GitHub Hugging Face ModelScope Ollama

技术报告 arXiv

与我们合作 面向 Agent 与推理服务的高速 token API、私有部署，以及模型与芯片的联合设计。欢迎来信 support@rockmatrix.ai。

作者 徐骅 张未名 季慧山 王意 陈歌 熊妍